

SYMMETRIC CRYPTOGRAPHY AND HASH FUNCTIONS

Symmetric cryptography is the **workhorse of modern data encryption** because it is exceptionally **fast and efficient**.

Symmetric-key cryptography concepts:

- **Shared Secret Keys** — are identical, confidential pieces of information (**keys**) used by two or more parties **to encrypt, decrypt, and authenticate data**, primarily in symmetric cryptography.
 - They ensure data confidentiality and integrity, with common examples including passwords, PINs, and API keys used in protocols like HMAC.
- **Block Cipher** — is a symmetric key algorithm that encrypts or decrypts data **in fixed-size chunks** (commonly 64 or 128 bits), rather than one bit at a time, using a secret key.
 - Key examples include:
 - **AES** (Advanced Encryption Standard)
 - **DES**
 - **Blowfish**
 - If the data isn't a multiple of the block size, it must be padded.
 - **Highly structured and widely used for general data encryption.**
- **Stream Cipher** — is a symmetric encryption algorithm that encrypts plaintext digits (**typically bits or bytes**) **individually, one at a time**, using a pseudorandom keystream from the secret key and combining it (usually via an XOR operation) with the plaintext.
 - It is designed for high-speed, low-latency applications like data-in-transit, real-time audio/video streaming, and secure, wireless communications, where **it is generally faster and less resource-intensive than block ciphers**.

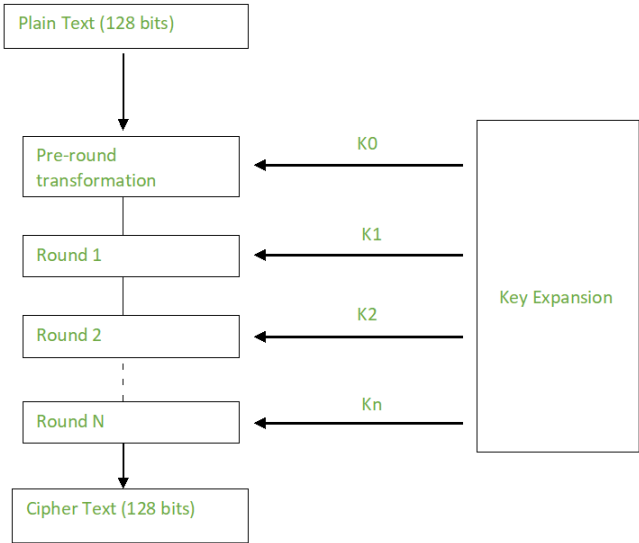
Advanced Encryption Standard (AES)

AES is the **global standard for symmetric encryption**. It is a **highly trusted encryption algorithm** used to secure data by converting it into an unreadable format without the proper key. It is a **block cipher with a fixed block size** of **128 bits**, regardless of the key size used.

- *That means it takes 128 bits as input and outputs 128 bits of encrypted cipher text.*
 - **performs operations on bytes of data rather than in bits.** **The number of rounds depends on the key length as follows:**
- **Key Sizes:**
 - **AES-128:** 10 rounds
 - **AES-192:** 12 rounds
 - **AES-256:** 14 rounds

Creation of Round Keys:

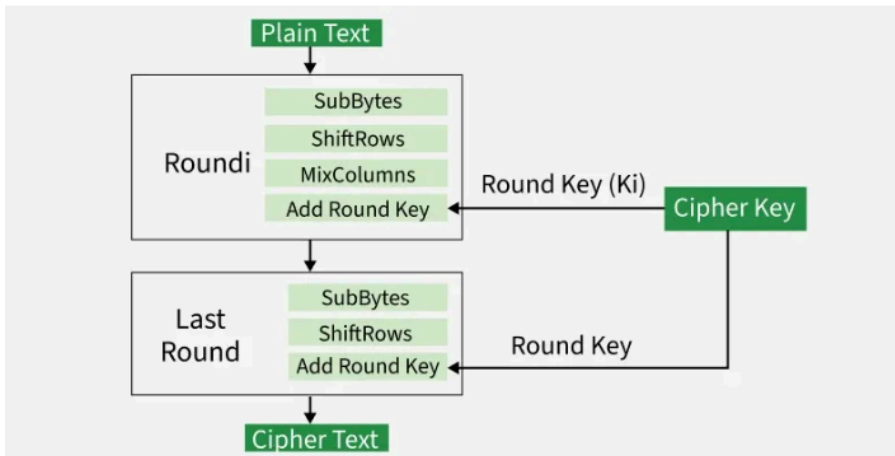
A Key Schedule algorithm calculates all the round keys from the key. So the initial key is used to create many different round keys which will be used in the corresponding round of the encryption.



Encryption:

AES considers each block as a 16-byte (4 byte x 4 byte = 128) grid in a column-major arrangement.

```
[ b0 | b4 | b8 | b12 |
| b1 | b5 | b9 | b13 |
| b2 | b6 | b10| b14 |
| b3 | b7 | b11| b15 ]
```



High-Level Structure:

AES is much stronger since it **does not use a Feistel network** (like older ciphers such as DES). Instead, it uses a **Substitution-Permutation Network (SPN)**.

- Every round of AES (except the final round) consists of **four distinct operations** designed to provide confusion (obscuring the relationship between the key and ciphertext) and diffusion (spreading the influence of one plaintext bit over many ciphertext bits):

1. **SubBytes (Confusion):** A non-linear substitution step where each byte is replaced with another according to a lookup table (the S-box).
 - This substitution is done in a way that a byte is never substituted by itself and also not substituted by another byte which is a complement of the current byte.
 - The result of this step is a 16-byte (4 x 4) matrix like before.

2. **ShiftRows (Diffusion):** A transposition step where the last three rows of the block's state matrix are shifted cyclically by different offsets.

This step is just as it sounds. Each row is shifted a particular number of times.

- The first row is not shifted
- The second row is shifted once to the left.
- The third row is shifted twice to the left.
- The fourth row is shifted thrice to the left.

(A left circular shift is performed.)

```
[ b0 | b1 | b2 | b3 ]      [ b0 | b1 | b2 | b3 ]
| b4 | b5 | b6 | b7 | -> | b5 | b6 | b7 | b4 |
| b8 | b9 | b10| b11|      | b10| b11| b8 | b9 |
| b12| b13| b14| b15|      | b15| b12| b13| b14|
```

3. **MixColumns (Diffusion):** A mathematical mixing operation that operates on the columns of the state matrix (matrix multiplication), combining the four bytes in each column. (This step is skipped in the final round).

$$\begin{bmatrix} c0 \\ c1 \\ c2 \\ c3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{bmatrix}$$

4. **AddRoundKey:** The subkey for the current round is combined with the state matrix using a bitwise XOR operation.
- Now the resultant output of the previous stage is XOR-ed with the corresponding round key. Here, the 16 bytes are not considered as a grid but just as 128 bits of data.
 - After all these rounds 128 bits of encrypted data are given back as output. This process is repeated until all the data to be encrypted undergoes this process.

Decryption:

The stages in the rounds can be easily undone as these stages have an opposite to it which when performed reverts the changes. Each 128 blocks goes through the 10,12 or 14 rounds depending on the key size.

The stages of each round of decryption are as follows :

- Add round key
- Inverse MixColumns
- ShiftRows
- Inverse SubByte

The decryption process is the encryption process done in reverse so I will explain the steps with notable differences.

Inverse MixColumns: This step is similar to the Mix Columns step in encryption but differs in the matrix used to carry out the operation.

- Mix Columns Operation each column is mixed independent of the other.
- Matrix multiplication is used. The output of this step is the matrix multiplication of the old values and a constant matrix.

$$\begin{aligned} [b0] &= [14 \ 11 \ 13 \ 9] [c0] \\ [b1] &= [9 \ 14 \ 11 \ 13] [c1] \\ [b2] &= [13 \ 9 \ 14 \ 11] [c2] \\ [b3] &= [11 \ 13 \ 9 \ 14] [c3] \end{aligned}$$

Inverse SubBytes: Inverse S-box is used as a lookup table and using which the bytes are substituted during decryption.

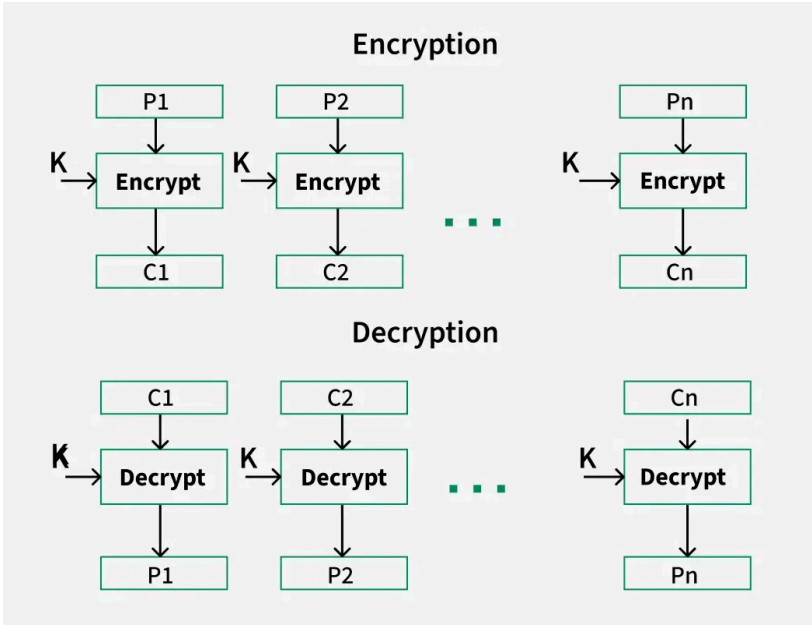
- Function Substitute performs a byte substitution on each byte of the input word. For this purpose, it uses an S-box.

Block Cipher Modes of Operation

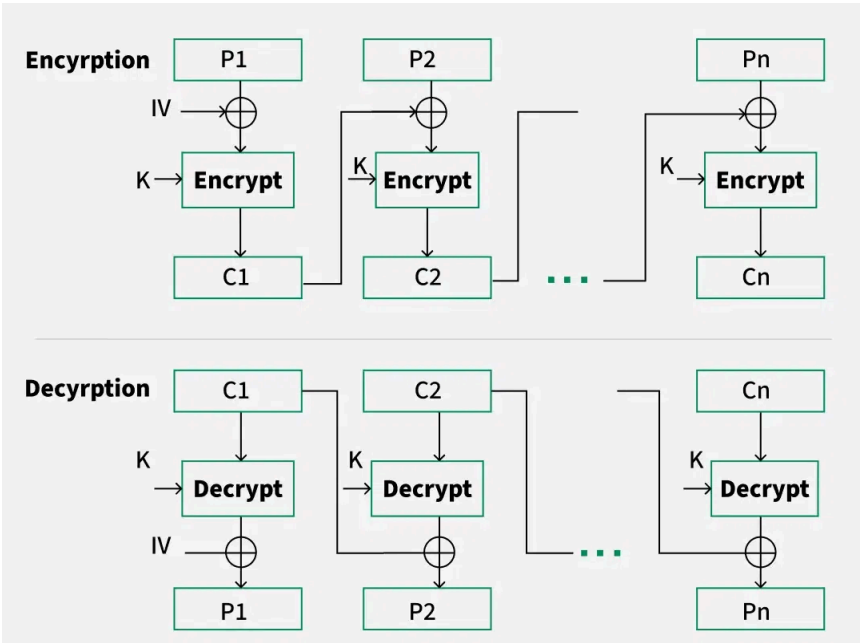
They define **how to securely encrypt and decrypt large amounts of data using a block cipher**. A block cipher is an **encryption algorithm** that processes data in fixed-size blocks (e.g., 128 bits) rather than one bit at a time. However, to encrypt data larger than a single block, different modes of operation are used to ensure both security and efficiency.

Since a block cipher like AES only encrypts 128 bits at a time, we need a **mode of operation** to **dictate how to encrypt data larger than a single block**.

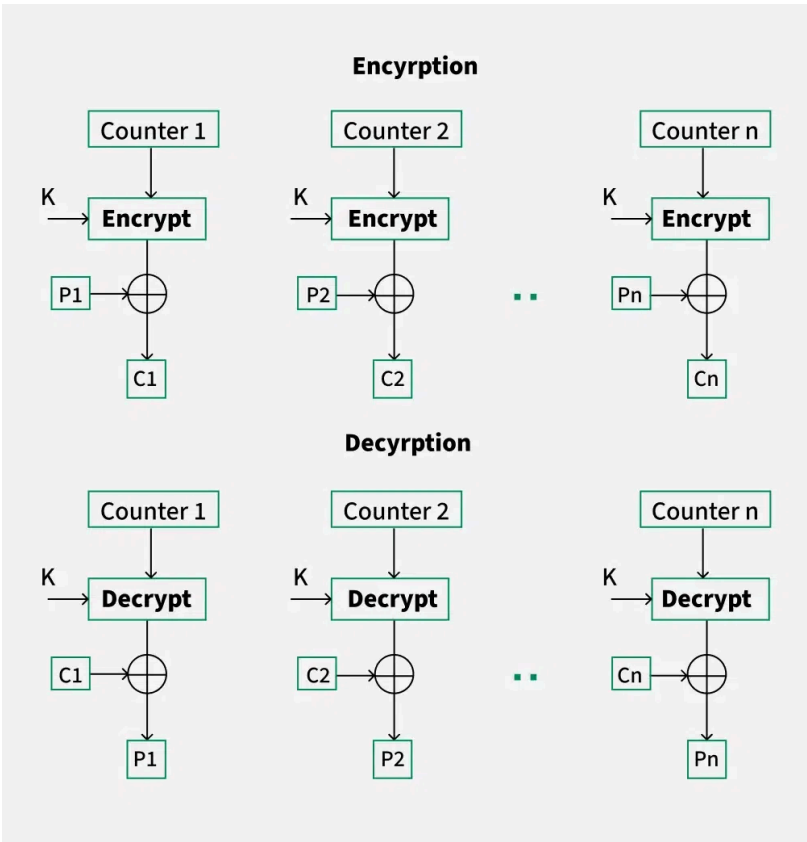
- 1. **Electronic Codebook (ECB)** — The most basic mode. Each block is encrypted independently using the same key. Generally, if a message is larger than b bits in size, it can be broken down into a bunch of blocks and the procedure is repeated.
 - **Why ECB is insecure:** Because it is deterministic, identical plaintext blocks always produce identical ciphertext blocks. If you encrypt an image in ECB mode, the visual patterns, contours, and shapes remain entirely visible in the ciphertext. It fails to hide data patterns and should almost never be used.



- 2. **Cipher Block Chaining (CBC)** — To fix ECB's pattern problem, CBC introduces **randomness**.
 - It requires an Initialization Vector (IV) for the first block.
 - Each plaintext block is XORed with the previous ciphertext block before being encrypted. This links (chains) the blocks together, meaning two identical plaintext blocks will yield different ciphertext blocks.



3. **Counter Mode (CTR)** — CTR effectively turns a block cipher into a stream cipher.
- Instead of encrypting the data directly, AES encrypts a sequential counter combined with a nonce (number used once).
 - This generates a keystream, which is then XORed with the plaintext. It is highly parallelizable (since blocks don't depend on previous blocks) and extremely fast.



4. **Galois/Counter Mode (GCM)** — GCM is an extension of CTR mode and is the modern industry standard (used heavily in TLS).
- It provides **Authenticated Encryption**. While modes like CBC and CTR provide confidentiality (hiding the data), they do not guarantee that the data hasn't been tampered with.
 - GCM calculates a cryptographic checksum (using Galois field mathematics) as it encrypts, ensuring both data privacy and data integrity/authenticity.

Cryptographic Hash Functions

A **hash function** takes an input (message) of any size and mathematically compresses it into a fixed-size string of bytes (the hash or digest).

Working of Cryptography Hash Function:

- **Input Processing:** Cryptographic hash functions process an input of any length—whether text, file, or data stream—and subject it to a sequence of mathematical operations. The input can range from several bytes to gigabytes of information.
- **Fixed-Size Output Generation:** No matter what the length of the input, the function generates a fixed-size hash value, normally in the form of a hexadecimal string. This uniform output size provides equality regardless of the inputs.
- **Deterministic Operation:** The hash function consistently computes the same hash for the same input. Such a property enables uncompromising data authentication, as any alteration in the input leads to a totally unique hash.
- **Avalanche Effect:** A minor alteration in the input, even the flipping of one bit, significantly alters the resultant hash. The sensitivity ensures that collisions among hashes (two inputs having the same hash) are highly unlikely.
- **One-Way Computation:** The algorithm is made irreversible in the sense that it is computationally impossible to recover the original input from its hash value. This one-way feature protects sensitive information such as passwords and digital signatures.

- **Collision Resistance:** Hash functions used in cryptography are designed to minimize the probability of two distinct inputs generating the same hash value, upholding the integrity and trustworthiness of verification processes.

Essential Properties:

For a hash function to be considered cryptographically secure, it must possess three traits:

1. **Preimage Resistance (One-way):** Given a hash output h , it should be computationally infeasible to figure out the original input m .
2. **Second-Preimage Resistance:** Given an input m_1 , it should be infeasible to find a different input m_2 that hashes to the exact same output.
3. **Collision Resistance:** It should be infeasible to find *any* two arbitrary inputs m_1 and m_2 that produce the same hash output.

Popular Cryptographic Hash Algorithms

MD5 (Message Digest Algorithm 5)

Once widely used for data integrity and digital signatures, MD5 is now considered insecure due to vulnerabilities that allow attackers to generate hash collisions easily. Its speed and simplicity made it popular historically, but it is no longer recommended for security-critical applications.

SHA-1 (Secure Hash Algorithm 1)

SHA-1 improved upon MD5 with a longer hash length and better resistance to collisions. However, advances in computational power and cryptanalysis exposed weaknesses, leading to practical collision attacks. Consequently, SHA-1 is deprecated for most security uses, including SSL/TLS certificates and digital signatures.

SHA-2 Family (SHA-256, SHA-512)

The SHA-2 family is currently the industry standard for cryptographic hashing, offering robust security with longer hash outputs of 256 and 512 bits. These algorithms provide strong collision and pre-image resistance, making them the preferred choice for secure communication protocols, blockchain technologies, and password hashing.

SHA-3 (Keccak)

Adopted as the latest NIST standard, SHA-3 uses a unique sponge construction different from SHA-2, enhancing security and flexibility. It offers comparable hash lengths with improved resistance to certain types of attacks, making it suitable for applications demanding long-term security.

BLAKE2 & BLAKE3

Designed as high-speed, secure alternatives to SHA-2 and SHA-3, BLAKE2 and BLAKE3 deliver faster hashing without compromising security. BLAKE3, in particular, supports parallel processing and incremental updates, making it ideal for modern systems requiring both speed and strong cryptographic guarantees.

SHA-256 vs. SHA-3

- **SHA-256 (Part of SHA-2):** Outputs a 256-bit hash. It is built on the **Merkle-Damgård construction**, which breaks the message into blocks and processes them through a compression function iteratively. It is currently the industry standard for most applications.
- **SHA-3:** The newest standard. It was designed to be fundamentally different from SHA-2 as a hedge against future cryptographic breakthroughs. It uses a **Sponge Construction** (absorbing data into a large state matrix, transforming it, and then squeezing out the hash).

Message Authentication Mechanisms

While hashes provide a unique fingerprint of data, anyone can alter the data and just generate a new hash. To prove that a message is both unmodified and came from a legitimate sender, we use authentication codes.

1. Message Authentication Code (MAC)

- A MAC is essentially a keyed hash. It takes the message and a shared secret key as inputs to produce a tag. Only someone possessing the secret key can generate or verify the MAC, thereby proving the message's origin and integrity.

2. Hash-based Message Authentication Code (HMAC)

- HMAC is a specific, highly secure way to construct a MAC using standard hash functions (like SHA-256) combined with a secret key. It was designed to resist specific cryptographic attacks (like length extension attacks) that can occur if you simply concatenate a key and a message.

The formal mathematical construction for HMAC is:

$$HMAC(K, m) = H((K \oplus opad) \parallel H((K \oplus ipad) \parallel m))$$

Where:

- H is the cryptographic hash function.
- K is the secret key.
- m is the message.
- $opad$ (outer padding) and $ipad$ (inner padding) are fixed hexadecimal constants.
- $\oplus$ denotes a bitwise XOR, and $\parallel$ denotes concatenation.